

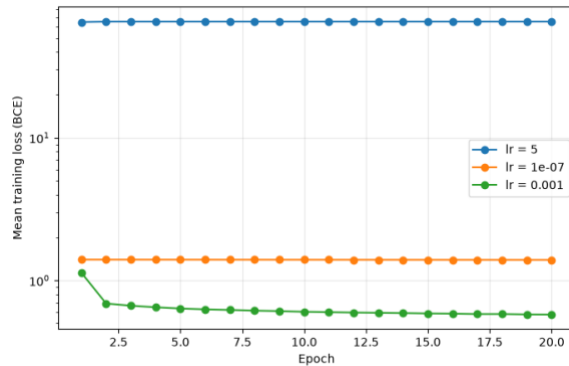
Assignment 05

Ethan Markow

July 14, 2026

Part 2: Hyperparameters

1. The weights are not hyperparameters since they are applied directly to the data, rather than describing the model itself



2.
 - a. A learning rate of 5 likely causes the model to overshoot the minimum of the loss function. A learning rate of 10^{-7} likely undershoots and never allows the gradient to descend fast enough to reach the minimum before the model runs out of epochs
3.
 - a. $\left\lceil \frac{768}{10} \right\rceil = 77$ weight updates per epoch
 - b. Over 100 epochs $77 \times 100 = 770$ weight updates
 - c. The last batch of each epoch is made up of 8 samples instead of 10 since 768 is not easily divisible by 10
4. If the batch size is set to 768, then every sample in the dataset is used to calculate the once in each epoch, this is known as batch gradient descent. If the batch size is 1, then only one sample is used to calculate the gradient at a time, 768 times per epoch, this is known as stochastic gradient descent.
5. There are two risks to this approach, if the gradient descent approaches the minimum well before 100 epochs have past, you are wasting time computing new weights and biases repeatedly to no gain. On the other hand, if 100 iterations are not enough to reach the minimum, then your model could be better at predicting and you would not have reached that capability. To proactively control the number of epochs, the code needs the error produced by each epoch and some error threshold you deem acceptable to stop iterating at.

6. The output must be a sigmoid here because the model is intended to choose between a binary yes/no target (whether the patient has diabetes or not), and the binary cross entropy loss function is intended for categorical targets.

Part 3: Mathematics

7. 7 input neurons $\times$ 12 layer 1 neurons + 12 layer 1 neurons $\times$ 8 layer 2 neurons + 8 layer 2 neurons $\times$ 1 output neuron + 12 layer 1 biases + 8 layer 2 biases + 1 output bias = 209 parameters

8.

a. $z = 0.1(0.5) - 0.2(1.0) + 0.3 = 0.15$

b. $\text{ReLU}(z) = 0.15$

c. $z = 0.1(0.5) - 0.2(1.0) - 0.3 = -0.45$ $\text{ReLU}(z) = 0$

$\frac{d(\text{ReLU}(z))}{dz} = \{0: z \leq 0, 1: z > 0\}$ this means that the gradient passed through this neuron becomes 0 and the weights and biases no longer update

9.

a. $\sigma(0) = \frac{1}{1+e^{-0}} = \frac{1}{1+1} = \frac{1}{2}$

b. $\sigma(0.15) = \frac{1}{1+e^{-0.15}} = 0.537$

- c. This is ideal for a probability distribution which must range from [0,1] across its domain

$$\lim_{z \rightarrow -\infty} \frac{1}{1 + e^{-z}} = 0$$

$$\lim_{z \rightarrow \infty} \frac{1}{1 + e^{-z}} = 1$$

10. $L = -(y \ln \hat{y} + (1 - y) \ln(1 - \hat{y}))$ $L(0.9,1) = 0.0458$ $L(0.1,1) = 1$ This shows how the closer to the 1/0 classifier the model is, the smaller the loss function is, enabling the gradient descent.

11.

a. $w_{new} = w_{old} - \eta \frac{\partial E}{\partial w} = 0.37 - 1.2(-0.0201) = 0.39412$

- b. A negative gradient indicates that increasing the weight will decrease the error function, and our goal is to find the minimum of the error function

12.

a. $\frac{\partial E}{\partial w} = \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$

b. $\frac{\partial z}{\partial w} = \frac{\partial}{\partial w} (\sum w_i x_i + b) = \sum x_i$ for a single weight w , this would just be x

c. $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = \sigma(z) - (\sigma(z))^2$ the maximum is $\sigma'(0) = 0.25$

13. After batch 3, each weight's stored gradient is equal to $g_1 + g_2 + g_3$ since they were never cleared before running the next batch. This leads to incorrect results since the new gradient, being used to update the weight, contains information from the previous batches
14. This would mean that the optimizer will try to update the weights before the gradient of the loss function is calculated for those weights and backpropagated through the model. Using an old gradient (or no gradient on the first iteration) to move the weights will result in incorrect results since the gradients are only meaningful to the weights they were calculated on. Since the overall gradient depends on all the weights, if any weight changes in the middle of calculating partial gradients, it makes the previous calculations in the batch invalid.